

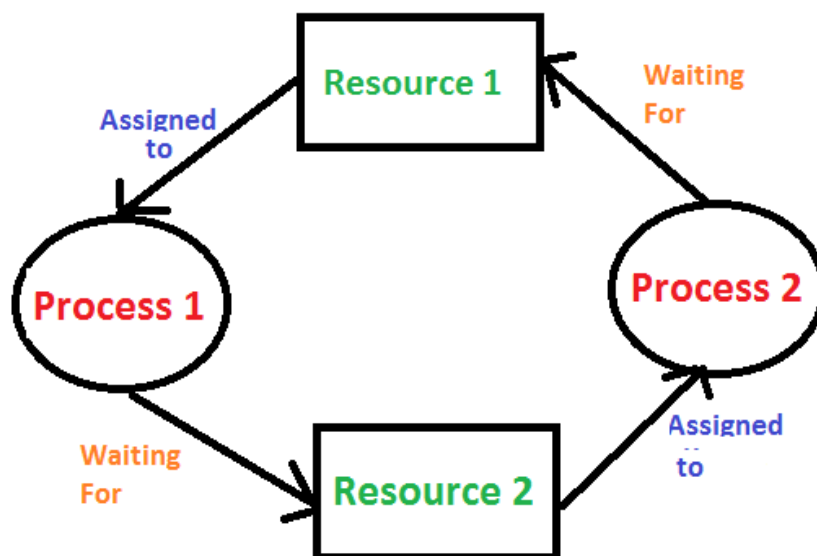
DEADLOCK

A **deadlock** is a situation in a multi-process system where a set of processes are **blocked permanently**, waiting for resources that are held by each other.

In other words, **each process in the set is waiting for a resource held by another process in the same set, and none of them can proceed.**

Example:

- Process P1 holds Resource R1 and waits for R2.
- Process P2 holds Resource R2 and waits for R1.
→ Both P1 and P2 are stuck forever — deadlock occurs.



How Does Deadlock Occur in OS?

A process in an operating system typically uses resources in the following sequence:

- Request a resource

- Use the resource
- Release the resource

Deadlock arises when processes hold some resources while waiting for others.

Example:

- Process P1 holds Resource R1 and requests R2.
- Process P2 holds Resource R2 and requests R1.
Neither process can proceed causing a deadlock.

How to visualize with a Resource Allocation Graph (RAG)

- Nodes: **Processes (P)** and **Resources (R)**
- Edges:
 - **Request edge:** $P \rightarrow R$ (process requests resource)
 - **Assignment edge:** $R \rightarrow P$ (resource assigned to process)

If the graph contains a **cycle**, deadlock may exist (if each resource has only one instance, cycle = deadlock).

Necessary Conditions for Deadlock in OS

Deadlock can arise if the following four conditions hold simultaneously (Necessary Conditions)

- **Mutual Exclusion:**

- At least one resource must be held in a non-shareable mode, meaning only one process can use it at a time.
- Example: A printer can only be used by one process at a time.

- **Hold and Wait:**

- A process holding at least one resource is waiting to acquire additional resources that are currently being held by other processes.
- Example: Process A holds a printer and waits for a scanner, while Process B holds a scanner and waits for the printer.

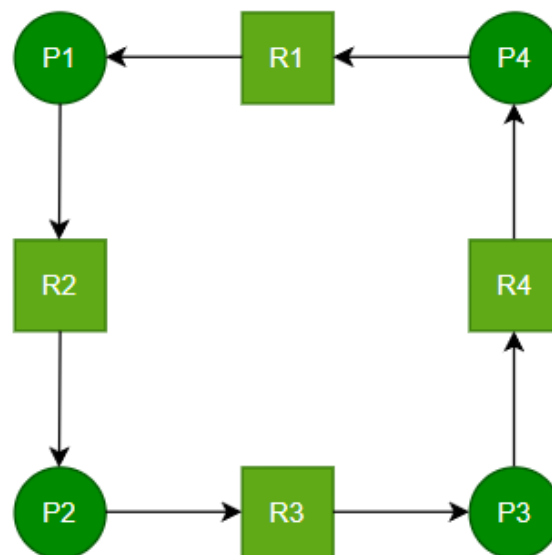
- **No Preemption:**

- Resources cannot be forcibly taken away from processes holding them. A process must release the resource voluntarily when it is no longer needed.
- Example: If Process A is holding a printer and is waiting for a scanner, the system cannot force Process A to give up the printer.

- **Circular Wait:**

- A set of processes exist such that each process in the set is waiting for a resource that another process in the set holds, forming a cycle of dependencies.
- Example: Process A waits for Process B's resource, Process B waits for Process C's resource, and Process C waits for Process A's resource, forming a circular chain.

EXAMPLE



The above image demonstrates a circular wait deadlock, here's how:

- **P1** is holding **R1** and waiting for **R2** (which is held by P2).
- **P2** is holding **R2** and waiting for **R3** (which is held by P3).
- **P3** is holding **R3** and waiting for **R4** (which is held by P4).

- **P4** is holding **R4** and waiting for **R1** (which is held by P1).

Methods of Handling Deadlock

Deadlocks in operating systems can be handled in four main ways:

1. **Deadlock Prevention** – Design the system so that at least one of the four necessary conditions (mutual exclusion, hold & wait, no preemption, circular wait) never occurs.

2. **Deadlock Avoidance** –

Deadlock avoidance mainly relies on algorithms that check resource allocation before making decisions. The two common types are:

- **Banker's Algorithm:** Used when multiple instances of resources exist. It simulates allocation and ensures that the system remains in a safe state before granting resources.
- **Resource Allocation Graph (RAG) Algorithm:** Used when each resource has only one instance. It checks for cycles in the graph to avoid unsafe states.

3. **Deadlock Detection & Recovery** – Allow deadlock to occur, then detect it using detection algorithms and recover (e.g., terminate a process or preempt resources).

It involves:

- **Process Termination:** Aborting one or more deadlocked processes to break the cycle.
- **Resource Preemption:** Temporarily taking resources from some processes and reallocating them to others.

4. **Ignore the Problem** – In some systems (like UNIX), deadlocks are rare, so the OS simply ignores them.